

CS 301: Distributed Systems - Lecture 8

Raft Consensus and Replicated State Machines

1. The Core Consensus Challenge

In a distributed architecture, independent servers must agree on values despite network partitions, packet loss, message delays, and server crashes. The fail-stop model assumes nodes fail cleanly rather than acting maliciously.

2. Replicated State Machine Architecture

Identical replica servers execute an identical sequence of deterministic commands from a shared replicated log. If each server starts in the same initial state, they will transition through identical intermediate states.

3. Three Decomposed Subproblems in Raft

Unlike Paxos, Raft explicitly decomposes consensus into three modules:

- A. Leader Election: Choosing one node to coordinate operations.
- B. Log Replication: The leader accepts writes and replicates them.
- C. Safety: Committed log entries must survive all future terms.

CS 301: Lecture 8 (Continued)

Server Roles, Quorum Math & Leader Completeness

4. Node State Machine & Logical Clock Terms

Each node exists in one of three states: Leader, Follower, or Candidate.

Time is divided into monotonically increasing terms acting as logical clocks.

If a message arrives with a higher term, any node steps down immediately.

5. Majority Quorum and Split-Brain Prevention

A candidate must acquire strictly greater than $N/2$ votes to become leader.

For example, in a 5-node cluster, quorum is at least 3 nodes.

Because any two majorities must overlap by at least one node, it is mathematically impossible for two leaders to be elected in the same term.

6. Randomized Election Timeouts (150ms - 300ms)

Randomizing election timeouts breaks split-vote deadlocks effectively.

Voters deny requests if the candidate's log is less up-to-date than their own.

7. Common Exam Trap: Disk Write vs Commit

A write to disk is NOT committed until replicated across a strict majority.